

IT

STÁTNICE

IT

Informační technologie

Architektura, infrastruktura a bezpečnost informačních technologií

Obsah všech otázek

01	Základní stavební prvky počítačů a jejich architektura
02	Architektura instrukční sady (ISA)
03	Přerušení a volání podprogramů
04	RISC procesory a proudové zpracování (Pipelining)
05	Hierarchie pamětí, Skrytá paměť (Cache) a Lokalita
06	Paralelismus v počítačových architekturách
07	Bezpečnostní architektura podniku (Monitorování, detekce, prevence)
08	Technická a organizační opatření pro bezpečnost IS a ochranu osobních údajů
09	Zabezpečení vnějšího perimetru a interních informačních systémů
10	Zabezpečení webové infrastruktury (Hrozby a prevence)

Základní stavební prvky počítačů a jejich architektura

Jaké jsou základní stavební prvky počítačů? Vysvětlete pojmy a popište základní funkce pro: Procesor – CPU, paměť, periférie, grafickou kartu (popište úlohu grafického procesoru GPU) a vysvětlete druhy a úlohu sběrnic. V současnosti se můžeme setkat s von Neumannovou a Harvardskou architekturou počítačů. Popište je. Jaké jsou jejich funkční bloky a jaká je jejich funkce? Co je instrukční cyklus počítače, jaké má fáze a jakou činnost procesor v těchto fázích vykonává?

1. Základní stavební prvky a jejich funkce

CPU (Central Processing Unit - Procesor)

“Mozek” počítače. Řídí činnost celého systému a vykonává instrukce programu. Skládá se z **ALU** (Aritmeticko-logická jednotka pro výpočty), **Řadiče** (řídí tok dat a dekoduje instrukce) a **Registrů** (nejrychlejší paměť přímo v CPU).

Paměť

Slouží k ukládání dat a instrukcí. Dělíme ji na **vnitřní/operační (RAM, ROM)**, se kterou CPU komunikuje přímo, a **vnější/sekundární (HDD, SSD)** pro dlouhodobé uložení dat.

Periférie (I/O zařízení)

Zařízení pro vstup (klávesnice, myš, senzory) a výstup (monitor, tiskárna, aktuátory) dat. Umožňují interakci počítače s okolním světem.

Grafická karta a GPU (Graphics Processing Unit)

Specializovaný koprocessor navržený pro rychlou manipulaci a změnu paměti za účelem urychlení tvorby obrazů v obrazovém vyrovnávacím poli (framebufferu).

Úloha GPU

Zpracování vysoce paralelních úloh (grafika, AI, strojové učení, vědecké výpočty). Má obrovské množství jednodušších jader oproti CPU (SIMD architektura).

Sběrnice (Buses)

Komunikační dálnice přenášející data mezi komponentami.

Druhy podle funkce

Datová sběrnice

Přenáší samotná data (obousměrná). Její šířka určuje propustnost (např. 64-bit).

Adresní sběrnice

Přenáší informaci o tom, *kam* se data posílají nebo *odkud* se čtou (jednosměrná z CPU). Šířka určuje max. velikost adresovatelné paměti.

Řídící sběrnice

Přenáší synchronizační a řídicí signály (Read, Write, Interrupt, Clock).

2. Architektury počítačů

Von Neumannova architektura

Tradiční koncept (1945), na kterém je postavena většina dnešních PC.

Princip

Data i instrukce programu jsou uloženy ve **společné (jednotné) paměti**.

Funkční bloky

- **ALU** provádí aritmetické a logické operace.
- **Řadič** načítá a dekoduje instrukce a řídí tok dat mezi bloky.
- **Společná paměť** uchovává instrukce i data.
- **I/O bloky** zajišťují komunikaci s periferiemi.
- **Sběrnice** propojuje CPU, paměť a vstupně-výstupní zařízení.

Nevýhoda (Von Neumannův úzký profil)

Z paměti nelze číst zároveň instrukci a data, což procesor zdržuje, protože využívají stejnou datovou sběrnici.

Harvardská architektura

Základ pro moderní mikrokontroléry, DSP procesory a interní cache CPU (L1).

Princip

Fyzicky **oddělená paměť** (a sběrnice) pro **data** a pro **instrukce**.

Funkční bloky

Podobné jako u von Neumannovy architektury, ale paměť a příslušné sběrnice jsou rozdělené:

- **Instrukční paměť a instrukční sběrnice** slouží pouze pro načítání programu.
- **Datová paměť a datová sběrnice** slouží pro čtení a zápis dat.
- **CPU** tak může současně načítat další instrukci a pracovat s daty předchozí instrukce.

Výhoda

Vyšší výkon – CPU může číst instrukci a zároveň přistupovat k datům v jednom hodinovém cyklu.

3. Instrukční cyklus počítače

Cyklus, kterým CPU zpracovává každou instrukci. Má tyto základní fáze (často zkracováno na *Fetch-Decode-Execute*):

Fetch (Výběr)

Řadič procesoru pošle adresu z registru PC (Program Counter) do paměti a načte strojový kód další instrukce do instrukčního registru (IR). PC se inkrementuje.

Decode (Dekódování)

Řadič analyzuje opkód (operační kód) v IR. Určí, jaká operace se má provést a kde se nacházejí potřebné operandy (adresní módy).

Execute (Vykonání)

ALU provede samotnou operaci (sčítání, logický součin, posun) nebo řadič provede skok/přesun dat.

4. (Volitelně) **Memory Access (Přístup do paměti)**: Pokud instrukce vyžaduje čtení/zápis dat z/do hlavní paměti, provede se nyní.
5. (Volitelně) **Write-back (Uložení výsledku)**: Výsledek operace je zapsán do cílového registru v CPU.

Praktické příklady

1. **Spuštění hry**: CPU řídí běh programu, RAM drží aktuální data hry, GPU vykresluje scénu, SSD dodává textury a sběrnice PCIe propojuje GPU se zbytkem systému. Když je pomalé SSD, hra se dlouho načítá; když je slabé GPU, klesají FPS.
2. **Instrukční cyklus na sčítání**: Instrukce `ADD R1, R2` se nejdříve načte z paměti, řadič ji dekoduje jako sčítání, ALU sečte hodnoty registrů a výsledek se zapíše zpět do cílového registru.

Architektura instrukční sady (ISA)

Architektura instrukční sady (Instruction Set Architecture) Co vše zahrnuje, s jakými typy se můžeme setkat a jaké mají vlastnosti? Jaké typické skupiny instrukcí najdeme v instrukčních sadách, jaké operace reprezentují a jak jsou kódovány? Uveďte příklady různých adresních módů využívaných v instrukcích.

1. Co zahrnuje ISA (Instruction Set Architecture)

ISA je **rozhraní (kontrakt) mezi hardwarem a softwarem**. Představuje programátorský model procesoru. Nevysvětluje, *jak* je procesor hardwarově postaven (mikroarchitektura), ale definuje, *jak* se *programuje*. Zahrnuje:

- Množinu podporovaných instrukcí a jejich nativní datové typy.
- Sadu a uspořádání viditelných registrů (GPR, PC, SP, stavové registry).
- Adresní prostor a adresní módy.
- Způsob obsluhy přerušení a výjimek.
- Kódování instrukcí (formát opkódu a operandů).

2. Typy ISA a jejich vlastnosti

CISC (Complex Instruction Set Computer)

- *Příklad: x86 (Intel, AMD).*

Vlastnosti

Proměnlivá délka instrukcí (např. 1–15 bytů). Instrukce mohou být velmi složité a provádět více operací najednou (např. aritmetická operace přímo s operandem v paměti).

Výhody

Menší velikost kódu (kompaktnější binárky), menší nároky na překladač.

Nevýhody

Složitější dekódování instrukcí v CPU, obtížnější pipelining.

RISC (Reduced Instruction Set Computer)

- *Příklad: ARM, MIPS, RISC-V.*

Vlastnosti

Pevná délka instrukcí (typicky 32 bitů). Instrukce jsou jednoduché, vykonávají se zpravidla v jednom hodinovém cyklu. Load/Store architektura (pouze instrukce Load a Store smí do paměti, ALU operace běží jen nad registry).

Výhody

Snadné a efektivní zřetězení (pipelining), menší a energeticky úspornější čipy.

Nevýhody

Delší výsledný kód (k provedení složité akce je potřeba více instrukcí).

3. Typické skupiny instrukcí a kódování

Instrukce jsou rozděleny do logických skupin podle typu operace:

Aritmeticko-logické (ALU)

ADD, SUB, AND, OR, XOR, posuny (SHL, SHR). Zpracovávají data.

Přesuny dat (Data Transfer)

MOV, LOAD, STORE, PUSH, POP. Přesunují data mezi registry navzájem, nebo mezi registry a pamětí.

Řízení toku (Control Flow)

JMP (nepodmíněný skok), JZ/JNZ (podmíněný skok dle příznaků), CALL, RET. Mění hodnotu Program Counteru (PC).

Systémové / Speciální

HALT, INT, instrukce pro synchronizaci (atomic compare-and-swap).

Kódování

Strojová instrukce v paměti je binární slovo složené z:

Opcode (Operační kód)

Určuje, co má procesor udělat (např. "sečti").

Operandy

Určují, s čím se má operace provést (registry, paměťové adresy, přímé hodnoty).

4. Adresní módy (Addressing Modes)

Adresní módy určují, **jakým způsobem procesor nalezne hodnotu operandu**.

Okamžitý (Immediate)

Hodnota operandu je přímo součástí instrukce.

- *Příklad:* `ADD R1, #5` (Přičti konstantu 5 k registru R1).

Registrový (Register)

Operand je přímo v zadaném registru CPU.

- *Příklad:* `MOV R1, R2` (Kopíruj obsah R2 do R1).

Přímý (Direct / Absolute)

Instrukce obsahuje přímo fyzickou adresu paměti.

- *Příklad:* `LOAD R1, [0x1000]` (Načti do R1 hodnotu z adresy 0x1000).

Nepřímý (Indirect)

Instrukce určuje registr, který obsahuje adresu paměti, na které jsou cílová data.

- *Příklad:* `LOAD R1, [R2]` (Načti do R1 hodnotu z paměti, jejíž adresa je v R2). Typické pro práci s ukazateli.

Indexovaný / Bázový (Indexed / Base-Offset)

K adrese v základním (base) registru se přičte offset (posun). Skvělé pro přístup k prvkům pole nebo strukturám.

- *Příklad:* `LOAD R1, [R2 + 4]` (Načti do R1 data z adresy R2+4).

Relativní k PC (PC-Relative)

Bázovým registrem je Program Counter. Používá se pro relativní skoky (nezávislé na absolutním umístění kódu v paměti – Position Independent Code).

- *Příklad:* `JMP [PC + 0x20]`.

Praktické příklady

1. **Stejný program, jiná ISA:** Kód v C typu `a = b + c` může běžet na x86 i ARM, ale překladač z něj vytvoří jiné strojové instrukce. Programátor vyšší úrovně vidí stejný výsledek, procesor ale vykonává instrukce podle své ISA.
2. **Indexované adresování u pole:** Pokud pole začíná na adrese v registru `R2` a jeden prvek má 4 byty, pak `LOAD R1, [R2 + 8]` načte třetí prvek pole. To je praktický důvod, proč adresní módy zjednodušují práci s poli a strukturami.

Přerušení a volání podprogramů

Existuje velmi silná podobnost mezi voláním podprogramu a vyvoláním obsluhy přerušení. Vysvětlete princip přerušení, jaké typy rozeznáváme a jak je implementováno. Jaké mohou být zdroje přerušení, jak se vybírá konkrétní zdroj přerušení k obslužení v daném okamžiku a jak jsou u obslužných podprogramů přerušení určeny jejich počátky?

1. Princip přerušení

Přerušení (Interrupt) je mechanismus, který umožňuje procesoru okamžitě reagovat na asynchronní nebo synchronní události bez nutnosti neustálého dotazování (pollingu).

Proces

Jakmile nastane požadavek na přerušení, CPU pozastaví vykonávání aktuálního programu.

- Uloží svůj stav (minimálně Program Counter a stavový registr) na **zásobník (stack)**.
- Procesor skočí na **obslužný podprogram (ISR - Interrupt Service Routine)**.
- Po dokončení ISR (instrukce `IRET`) obnoví stav ze zásobníku a pokračuje přesně tam, kde přestal.

Podobnost s podprogramem (`CALL`)

Oba mechanismy přeruší lineární tok kódu, uloží návratovou adresu na zásobník a skočí na jiný kód.

Rozdíl: Volání podprogramu je *synchronní* (vyžádané programátorem v kódu). Přerušení může přijít *kdykoliv* (asynchronně od HW) a program o něm neví.

2. Typy a zdroje přerušení

Hardwarová přerušení (Externí)

Vyvolaná periferiemi (např. stisk klávesy, přijetí paketu síťovou kartou, tik hardwarového časovače).

Maskovatelná (INTR)

Procesor je může dočasně ignorovat (maskovat instrukcí typu `CLI`).

Nemaskovatelná (NMI)

Kritické události (selhání paměti, pokles napětí), které procesor musí obsloužit vždy.

Softwarová přerušení (Interní / Výjimky)

Výjimky (Exceptions)

Vznikají jako chyba při běhu instrukce (dělení nulou, neplatný opkód, výpadek stránky paměti - page fault).

Programová přerušení

Vyvolaná explicitně instrukcí (např. `INT 0x80`). Typicky se využívají k volání služeb operačního systému (System Calls) – bezpečný přechod z User mode do Kernel mode.

3. Výběr zdroje přerušení a priorit

Pokud dojde k vícero požadavkům na přerušení současně, je nutné určit, které se zpracuje první.

- O to se stará **Řadič přerušení (PIC - Programmable Interrupt Controller nebo modernější APIC)**.
- Každý zdroj přerušení (IRQ) má přiřazenou **prioritu**.
- PIC vybere to s nejvyšší prioritou a pošle procesoru odpovídající číslo (vektor přerušení). Nižší priority jsou pozdrženy.

4. Určení počátků obslužných podprogramů

Aby CPU věděl, *kam* má přesně skočit pro konkrétní událost, využívá mechanismus vektorování.

Vektor přerušení

Číslo (často 8bitové, 0-255), které identifikuje konkrétní událost.

Tabulka vektorů přerušení (IVT - Interrupt Vector Table, u x86 IDT)

Datová struktura umístěná pevně v paměti. Obsahuje adresy (ukazatele) na samotné rutiny (ISR).

Postup

CPU dostane od řadiče vektor (např. 4). Vektor slouží jako **index do této tabulky**. CPU si z tabulky přečte adresu a skočí na ni.

Praktické příklady

1. **Stisk klávesy:** Procesor právě provádí jiný program, ale klávesnice vyvolá hardwarové přerušení. CPU uloží stav, skočí do obsluhy klávesnice, přečte znak a potom se vrátí zpět k původnímu programu.
2. **Systémové volání:** Aplikace chce přečíst soubor, ale sama nesmí přímo pracovat s diskem. Vyvolá softwarové přerušení nebo instrukci `syscall`, CPU přejde do kernel mode a operační systém bezpečně provede operaci.

RISC procesory a proudové zpracování (Pipelining)

Procesory pro tablety a mobilní telefony typicky obsahují procesory typu RISC, které s výhodou využívají proudové zpracování instrukcí. Vysvětlete, jak se tyto procesory liší od procesorů typu CISC. Jaký je princip proudového zpracování, jak se jím dosahuje vysokého výkonu a co jej může narušit?

1. Rozdíly mezi procesory RISC a CISC

Procesory pro mobilní zařízení (ARM) typicky využívají architekturu RISC, která je energeticky a výpočetně efektivnější než tradiční CISC (typicky desktopový x86/x64).

RISC (Reduced Instruction Set Computer)

- Jednoduché instrukce, pevná délka (např. 32 bitů).
- Jedna instrukce = jeden hodinový takt (ideálně).

Load/Store architektura

Pouze instrukce Load a Store smí pracovat s pamětí RAM. Všechny aritmetické a logické operace probíhají výhradně nad vnitřními registry.

- Obsahuje velký počet univerzálních registrů (GPR).
- *Výhoda:* Snadná implementace proudového zpracování, malá spotřeba, malá plocha čipu.

CISC (Complex Instruction Set Computer)

- Složité instrukce, proměnlivá délka (např. 1–15 bytů).
- Instrukce může trvat mnoho taktů a přímo pracovat s pamětí (např. `ADD [mem], R1`).
- Má menší počet registrů, komplikovanější dekodér.

2. Princip proudového zpracování instrukcí (Pipelining)

Pipelining je technika paralelizace na úrovni instrukcí (ILP), která radikálně zvyšuje výkon (propustnost) procesoru.

Princip

Instrukční cyklus (např. zpracování instrukce v 5 krocích: *Fetch, Decode, Execute, Memory, Write-back*) se rozdělí na **nezávislé stupně (fáze)** procesoru.

- Zatímco jedna instrukce je vykonávána (Execute), následující instrukce se již dekoduje (Decode) a další se vybírá z paměti (Fetch).

- Chová se jako **montážní linka v továrně**.

Jak dosahuje výkonu

Ačkoliv zpoždění (latence) jedné instrukce zůstává stejné, roste **propustnost (Throughput)**. Při ideálním zaplnění roury (pipeline) dokončí CPU jednu instrukci **každý hodinový cyklus**.

3. Co může proudové zpracování narušit (Hazardy)

Ideální stav naráží na tzv. hazardy, které způsobí, že se pipeline musí pozastavit (tzv. **pipeline stall** nebo **bubble**).

Datový hazard

- *Příčina:* Instrukce potřebuje pro svůj běh výsledek předchozí instrukce, která ale ještě nedošla do fáze zápisu výsledku. (Např. závislost Read-After-Write).
- *Řešení:* **Forwarding (Bypassing)** – hardware přesměruje výsledek z fáze Execute rovnou do vstupu ALU pro další instrukci, bez čekání na zápis do registru.

Řídící (Skokový) hazard

- *Příčina:* Instrukce podmíněného skoku. Procesor neví, zda se skok provede nebo ne, a tak neví, kterou instrukci má do "roury" načíst jako další. Pokud načte špatnou větev, musí celou pipeline po vyhodnocení skoku "spláchnout" (flush), což je obrovská ztráta výkonu.
- *Řešení:* **Predikce skoků (Branch Prediction)** – sofistikované HW obvody, které se snaží uhodnout, kam program skočí (např. smyčky se většinou opakují).

Strukturální hazard

- *Příčina:* Dvě různé instrukce v různých fázích pipeline vyžadují přístup ke stejnému HW prostředku (např. současné čtení dat a instrukce z paměti po jedné sběrnici).
- *Řešení:* Replikace HW komponent – typicky zavedení **Harvardské architektury** na úrovni cache (oddělená L1 Data cache a L1 Instruction cache).

Praktické příklady

1. **RISC Load/Store:** Na ARM procesoru se hodnota z paměti nejdříve načte instrukcí **LOAD** do registru, potom se provede **ADD** nad registry a až výsledek se uloží zpět instrukcí **STORE**. Díky jednoduchým instrukcím se pipeline navrhuje snadněji.
2. **Branch prediction ve smyčce:** V cyklu **for** procesor předpokládá, že se skok na začátek smyčky bude opakovat. Pokud předpověď vyjde, pipeline běží plynule; při poslední iteraci se předpověď netrefí a pipeline se musí částečně zahodit.

Hierarchie pamětí, Skrytá paměť (Cache) a Lokalita

Výkon počítače může výrazně ovlivnit hierarchie paměťového podsystemu. Vysvětlete strukturu a funkci jednotlivých komponent. Jak ovlivňuje přítomnost skrytých pamětí (cache) výkon počítače, co je časová a prostorová lokalita? Jaká je konstrukce skryté paměti? Je výhodnější nižší nebo vyšší stupeň asociativity? Jak mohou instrukce prefetch, které nahrávají data do skryté paměti na žádost programátora, snížit počet výpadků?

1. Hierarchie paměťového podsystemu

Procesor je mnohem rychlejší než hlavní paměť (RAM). Aby CPU nečekalo na data, staví se paměti do hierarchie (pyramidy), kde platí: **čím blíže k CPU, tím je paměť rychlejší, menší a dražší.**

Registry (v CPU)

Běží na frekvenci procesoru, kapacita bajty, nejrychlejší.

L1, L2, L3 Cache (SRAM)

Uvnitř čipu CPU. Extrémně rychlé (nanosekundy), kapacita kB až MB.

Operační paměť RAM (DRAM)

Hlavní paměť, stovky nanosekund latence, kapacita GB.

Sekundární úložiště (SSD, HDD)

Permanentní paměť. Nejpomalejší (mikrosekundy až milisekundy), obrovská kapacita (TB).

2. Časová a prostorová lokalita referencí

Celý koncept zrychlení pomocí mezipamětí funguje díky tomu, jak jsou reálně psané programy. Přístup k paměti není náhodný.

Časová lokalita (Temporal locality)

Přistoupí-li program k datové položce v paměti, je vysoce pravděpodobné, že k téže položce přistoupí brzy znovu (např. proměnná iterátoru v cyklu `for`).

Prostorová lokalita (Spatial locality)

Přistoupí-li program k datové položce na určité adrese, je velmi pravděpodobné, že brzy přistoupí i k položkám, které se nacházejí v její těsné fyzické blízkosti (např. sekvenční čtení prvků pole (array) za sebou).

3. Vliv a konstrukce Skryté paměti (Cache)

Vliv na výkon: Cache ukládá kopie často (nebo nedávno) používaných dat z RAM. Když CPU potřebuje data, hledá je nejprve v Cache. Pokud je najde (**Cache Hit**), získá je okamžitě. Pokud ne (**Cache Miss**), musí se přenést blok dat z pomalé RAM do Cache, což CPU zdržuje (latence). Vysoký *Hit Rate* (> 95%) radikálně zvyšuje reálný výkon počítače.

Pro dosažení prostorové lokality se nepřenáší bajty, ale celé **bloky (Cache Lines)** – typicky 64 B.

Konstrukce cache a asociativita

Jak mapujeme blok z RAM do Cache?

Přímo mapovaná (Direct Mapped)

Blok paměti RAM má určeno právě jedno jediné místo v Cache (určeno podle modulo adresa). Rychlé hledání, ale hrozí masivní "vyhazování" dat (thrashing), pokud program potřebuje dvě různé adresy mapované do stejného slotu v Cache.

Plně asociativní (Fully Associative)

Blok RAM lze uložit do jakéhokoli volného místa v Cache. Řeší kolize, ale prohledávání všech míst (komparátory pro každý slot) je hardwarově extrémně drahé a pomalé pro velkou kapacitu.

N-cestně asociativní (N-way Set Associative)

Kompromis. Cache je rozdělena na sady (sets). Každá množina má "N" cest (slotů). Blok z RAM má pevnou množinu, ale uvnitř ní si může vybrat jakoukoliv z N cest. Běžně bývá 4, 8 nebo 16-cestná.

Je výhodnější nižší nebo vyšší asociativita?

Vyšší asociativita

(např. 16-cestná) snižuje počet *Cache Miss* způsobených kolizemi. Daň za to je složitější hardware, více energie a mírně delší čas hledání (Hit Time).

- **Závěr:** Vyplatí se kompromis, v praxi L1 cache (musí být blesková) bývá méně asociativní, zatímco L3 cache (hlavní cíl je zabránit Miss do RAM) bývá vysoce asociativní.

4. Instrukce Prefetch

Prefetch (přednačítání)

Softwarová instrukce (nebo automatický hardwarový algoritmus CPU), která slouží k nažádání přenosu dat z hlavní paměti do Cache *ještě předtím*, než o ně program reálně požádá (např. procesor počítá 1. prvek pole, a programátor tam vloží `prefetch` pro prvek 20.).

Důsledek

Překrývá latenci paměti. Než program dopracuje k datům, data už "čekají" připravena v L1/L2 cache, tím pádem klesne počet *Cache Miss* a program se nemusí zastavovat a čekat na RAM.

Praktické příklady

- 1. Pole vs. spojový seznam:** Sekvenční průchod polem je rychlý, protože prvky leží vedle sebe v paměti a cache načte celý cache line. Spojový seznam může skákat po paměti, takže častěji vznikají cache misses.
- 2. Prefetch u zpracování videa:** Program ví, že za chvíli bude zpracovávat další blok pixelů. Pošle prefetch pro budoucí blok dat, takže se data mezitím načtou do cache a CPU méně čeká na RAM.

Paralelismus v počítačových architekturách

V současných počítačových architekturách se setkáváme s paralelismem na mnoha úrovních. Od paralelismu na úrovni instrukcí až po víceprocesorové systémy. Vysvětlete, jaké typy paralelismu lze nalézt ve skalárních, superskalárních, vícevláknových a vícejádrových a VLIW procesorech. Popište tyto architektury a co, případně kdo, provádí paralelizaci. Jaký vliv má optimalizace kódu překladačem na dosažení maximálního reálného výkonu?

1. Úrovně paralelismu

ILP (Instruction Level Parallelism)

Paralelismus na úrovni instrukcí. Schopnost vykonávat více nezávislých instrukcí jednoho vlákna současně.

TLP (Thread Level Parallelism)

Paralelismus na úrovni vláken. Souběžné vykonávání instrukcí pocházejících z různých vláken nebo procesů.

DLP (Data Level Parallelism)

Zpracování stejné operace nad množinou různých dat najednou (SIMD – Single Instruction, Multiple Data, např. vektorové instrukce AVX).

2. Architektury a zpracování paralelismu

Skalární procesory

- Zpracovávají maximálně jednu instrukci v jednom hodinovém taktu (alespoň ve fázi Execute).
- Základní formou zrychlení je **proudové zpracování (Pipelining)**.
- *Paralelizace*: Provádí hardware (překrývání fází instrukcí).

Superskalární procesory

- Obsahují v jednom jádře **více prováděcích jednotek** (např. 2x ALU, 2x FPU). Můžou v jednom taktu načíst, dekodovat a vykonat více instrukcí (využívají ILP).
- Při zpracování programu (Out-of-Order execution) hardware dynamicky hledá instrukce, které na sobě datově nezávisí, a pošle je do volných jednotek.
- *Paralelizace*: Zcela dynamicky a transparentně **provádí hardware (Scheduler/Dispatcher)** procesoru během běhu.

VLIW (Very Long Instruction Word)

- Alternativa k superskalárům, ale s mnohem jednodušším hardwarem. Místo hardwarového hledání nezávislých instrukcí posílá do procesoru rovnou obří instrukční slovo, které obsahuje více nezávislých operací (např. "udělej ADD na ALU1, a MUL na FPU").
- *Paralelizace*: Tuto těžkou analýzu závislostí a paralelní zabalení **musí provést překladač (Kompilátor)** staticky již při překladu kódu. Pokud to překladač nedokáže, nahradí zbytek slova instrukcemi `NOP` (prázdná operace) a výkon rapidně klesá.

Vícevláknové procesory (Multithreading / SMT - Hyper-Threading)

- Fyzické jádro má replikovanou architekturu stavu (registry, PC), ale sdílí prováděcí jednotky. Pro OS se jedno fyzické jádro tváří jako dvě (nebo více) logických jader.
- Když jedno vlákno čeká na data z RAM, jádro beze ztráty taktu přepne na vykonávání instrukcí druhého vlákna, čímž plní jinak nevyužitý ALU.
- *Paralelizace*: Využívá TLP. Vlákna vytváří **programátor a OS**. Hardware spravuje jejich souběžné "krmení" do pipeline.

Vícejádrové a víceprocesorové systémy (Multi-core)

- Více fyzických jader na jednom čipu nebo na základní desce. Skutečný TLP.
- *Paralelizace*: Zcela závislá na **programátorovi** (psaní vícevláknových aplikací, zámky, synchronizace) a **operačním systému** (přidělování procesů jádrům).

3. Vliv překladače (kompilátoru) na výkon

Překladač má obrovský vliv na využití potenciálu hardwaru. Napsaný kód nemusí být pro moderní CPU optimální.

U VLIW

Jak bylo zmíněno, překladač je naprosto klíčový. Bez agresivní statické paralelizace překladačem architektura selhává.

U superskalárů a skalárů

Překladač provádí optimalizace jako:

Instruction Scheduling (plánování instrukcí)

Změna pořadí instrukcí v kódu tak, aby hardware co nejméně narážel na datové hazardy (oddálení závislých instrukcí od sebe).

Loop Unrolling (rozbalování cyklů)

Zmenšuje počet řídicích hazardů (skoků) a zvětšuje blok lineárního kódu, ve kterém může superskalární HW hledat ILP paralelizaci.

- Bez kvalitního překladače by i ten nejlepší hardware nevyužil všechny své jednotky (nízké IPC - Instructions Per Cycle).

Praktické příklady

- 1. Superskalární CPU:** Procesor může v jednom taktu provést nezávislé instrukce, například jednu celočíselnou operaci a jednu operaci s desetinnými čísly, pokud mezi nimi není datová závislost. Programátor to často přímo neřídí, hledá to hardware.
- 2. Vícejádrový web server:** Webový server obsluhuje mnoho požadavků najednou. Operační systém rozděluje vlákna na více jader, takže jeden požadavek může čekat na databázi a jiný se mezitím zpracovává na jiném jádře.

Bezpečnostní architektura podniku (Monitorování, detekce, prevence)

Představte si, že jste odpovědný za bezpečnost infrastruktury podniku. Navrhněte a popište architekturu, která bude monitorovat útoky na tuto infrastrukturu, detekovat je a bude jim zabráňovat. Nezapomeňte zmínit metody zabezpečení na aplikační vrstvě.

Při návrhu bezpečnostní architektury podniku vycházíme z principu **Defense in Depth (Hloubková obrana)** – vrstveného zabezpečení, kde selhání jedné vrstvy neznamená pád celého systému. Zahrnuje fáze: ochrana (prevence), detekce, reakce.

1. Monitorování a detekce (Odhalení útoků)

Cílem je mít kompletní přehled o dění v síti a včas odhalit anomálie.

IDS (Intrusion Detection System)

- Systém pro detekci průniků. Pasivně analyzuje síťový provoz (NIDS – kopie provozu ze switchů) nebo aktivitu na koncových stanicích (HIDS).
- Využívá **signatur** (databáze známých útoků) a **analýzy chování** (odchylky od běžného stavu). Generuje logy a upozornění.

EDR / XDR (Endpoint / Extended Detection and Response)

- Pokročilé řešení na koncových bodech (serverech, stanicích). Sleduje spouštění procesů, změny v registrech a anomální chování. Detekuje ransomware a fileless malware.

SIEM (Security Information and Event Management)

- Centrální "mozek" bezpečnostního dohledu. SIEM sbírá logy ze všech systémů v podniku (firewally, Active Directory, servery, databáze).
- Provádí **korelaci událostí** (např. spojí "5x chybné heslo na VPN" a následně "přihlášení z neznámé země" do jednoho kritického incidentu).

SOC (Security Operations Center)

Tým bezpečnostních analytiků, kteří neustále (24/7) vyhodnocují alerty ze SIEMu a inicializují reakci.

2. Zabránění útokům (Prevence / Ochrana perimetru a sítě)

Cílem je zastavit útok ještě předtím, než zasáhne chráněné systémy.

NGFW (Next-Generation Firewall)

- Chrání hranici mezi interní sítí a internetem i mezi interními segmenty (VLAN). Neřídí se jen porty (L3/L4), ale umí identifikovat aplikace a hrozby.

IPS (Intrusion Prevention System)

- Na rozdíl od IDS je umístěn přímo v toku dat (inline). Pokud detekuje škodlivý paket, **aktivně ho zahodí** nebo zablokuje IP adresu útočníka.

Zero Trust Architecture (ZTA)

- Princip „nikomu nevěř, všechno ověř“. Každý přístup (i z vnitřní sítě) musí být silně autentizován a autorizován (mikrosegmentace).

NAC (Network Access Control)

Zajišťuje, že se do sítě připojí jen schválená a „zdravá“ zařízení (ověření přítomnosti antiviru, aktualizací OS atd.).

3. Zabezpečení na aplikační vrstvě (L7)

Klasický firewall nezabrání útokům, které jdou na povolené porty webových aplikací (80/443). Proto je nutná ochrana přímo na L7.

WAF (Web Application Firewall)

- Specializovaný proxy firewall stojící před webovými servery. Rozumí HTTP/HTTPS komunikaci.
- Blokuje nejčastější zranitelnosti z OWASP Top 10, např.: **SQL Injection**, **XSS (Cross-Site Scripting)**, pokusy o přetečení bufferu nebo neobvykle dlouhé URL požadavky.

Silná autentizace a řízení relací

- Zabezpečení proti prolomení hesel (Brute-force) pomocí vynucení **Vícefázového ověřování (MFA)**, zámku účtů po více pokusech a používání bezpečných protokolů pro federovanou identitu (OAuth 2.0 / OIDC, SAML). Ochrana session cookies (příznaky `HttpOnly`, `Secure`).

Ochrana in-transit dat (Šifrování)

Vynucení moderních verzí TLS (1.2 / 1.3) a HSTS (HTTP Strict Transport Security) pro ochranu proti odposlechu (MitM útokům).

DevSecOps a SAST/DAST

Začlenění bezpečnostních testů kódu aplikací přímo do CI/CD pipeline už během vývoje.

Praktické příklady

1. **SIEM korelace:** SIEM uvidí 20 neúspěšných přihlášení na VPN, následně úspěšné přihlášení z cizí země a potom pokus o přístup k účetnímu serveru. Jednotlivě by to nemuselo stačit, ale dohromady jde o podezřelý incident.

- 2. Defense in Depth:** Útok na web nejdřív filtruje WAF, přístup administrátora chrání MFA, server monitoruje EDR a síťové logy sbírá SIEM. Když jedna vrstva selže, další vrstvy stále mohou útok detekovat nebo zastavit.

Technická a organizační opatření pro bezpečnost IS a ochranu osobních údajů

Pokud byste byl odpovědný za bezpečnost informačního systému v organizaci, jaká technická a organizační opatření byste své organizaci navrhl, aby byla zajištěna dostatečná úroveň bezpečnosti a ochrany osobních údajů zpracovávaných v informačním systému?

Návrh opatření musí zajistit tzv. **CIA triádu** (Důvěrnost, Integrita, Dostupnost) a zajistit plný soulad s legislativou (zejména **GDPR** a Zákon o kybernetické bezpečnosti). Opatření dělíme na technická (řešená pomocí HW/SW) a organizační (procesy a lidé).

1. Technická opatření

Týkají se samotných informačních systémů a infrastruktury organizace.

Řízení přístupu (Access Control)

- Zavedení principu **nejnižšího privilegia (PoLP)** – uživatelé mají přístup pouze k těm osobním údajům, které nezbytně potřebují ke své práci (RBAC – Role-Based Access Control).
- Vynucení **vícefázového ověřování (MFA)** pro přístup do systému, obzvláště zvenčí (VPN) a pro administrátorské účty.

Šifrování dat (Kryptografie)

- *Data in Transit*: Použití silného šifrování pro veškerou síťovou komunikaci (TLS 1.2/1.3 pro web, IPsec/OpenVPN pro vzdálený přístup).
- *Data at Rest*: Šifrování pevných disků (BitLocker, LUKS) na noteboocích a serverech, šifrování sloupců v databázích obsahujících citlivá data.

Pseudonymizace a Anonymizace (dle GDPR)

Náhrada identifikačních údajů (např. rodných čísel) v testovacích či analytických databázích za umělé identifikátory, čímž se snižuje riziko při případném úniku.

Zálohování a obnova (Business Continuity / Disaster Recovery)

- Pravidelné, šifrované automatické zálohy.
- Využití pravidla 3-2-1 (3 kopie dat, na 2 různých médiích, z toho 1 mimo hlavní lokalitu/v cloudu chráněná proti ransomwaru). Pravidelné testování obnovy.

Zabezpečení koncových bodů a sítě

Nasazení Next-Gen Firewallů (segmentace databázových serverů od běžných uživatelů) a EDR (Endpoint Detection and Response) antivirových řešení na stanice.

Auditování a logování

Zaznamenávání přístupů k osobním údajům (kdo, kdy, co prohlížel/upravoval) do nezměnitelného logu (zajištění odpovědnosti – Accountability).

2. Organizační opatření

Týkají se lidí, pravidel a administrativních procesů uvnitř organizace (lidský faktor je často nejzranitelnějším místem).

Bezpečnostní politiky a směrnice

- Vytvoření interních dokumentů schválených vedením (Politika hesel, Politika vzdáleného přístupu, Clean Desk / Clear Screen Policy – nenechávat citlivé dokumenty na stole a zamykat obrazovky).

Školení a bezpečnostní povědomí (Security Awareness)

- Pravidelná, povinná školení zaměstnanců. Zaměření na rozpoznání metod sociálního inženýrství (Phishing, Baiting) a na správné nakládání s osobními údaji podle GDPR.

Proces řízení bezpečnostních incidentů (Incident Response Plan)

- Jasně definovaný postup, jak reagovat na únik dat. Kdo vyhodnocuje závažnost, kdo komunikuje s veřejností.
- Proces musí zajistit schopnost ohlásit závažný únik osobních údajů dozorovému orgánu (ÚOOÚ) **do 72 hodin**, jak požaduje GDPR.

Osoba pověřence pro ochranu osobních údajů (DPO)

- Jmenování DPO (pokud to organizaci ukládá GDPR), který nezávisle dohlíží na soulad procesů v organizaci s legislativou a komunikuje s úřady.

Řízení dodavatelů (Smlouvy se zpracovateli)

- Uzavírání platných smluv o zpracování osobních údajů (DPA – Data Processing Agreement) se všemi externími poskytovateli (např. cloudové služby, účetní firmy), aby garantovali minimálně stejnou úroveň ochrany dat.

Pravidelné auditů a penetrační testy

Zajištění externího posouzení (etický hacking, audit), které ověří funkčnost technických a organizačních opatření v praxi.

Praktické příklady

1. **Ochrana mzdové databáze:** Přístup mají jen personalisté podle RBAC, administrátor používá MFA, databáze je šifrovaná a všechny přístupy k rodným číslům se logují. To kombinuje technická opatření

pro důvěrnost i odpovědnost.

2. **Únik osobních údajů:** Zaměstnanec omylem odešle export zákazníků na špatný e-mail. Organizace podle incident response plánu vyhodnotí rozsah, informuje DPO, zajistí nápravná opatření a u závažného incidentu řeší ohlášení ÚOOÚ do 72 hodin.

Zabezpečení vnějšího perimetru a interních informačních systémů

Pomocí jakých prostředků a opatření byste provedl zabezpečení vnějšího perimetru (Internet, veřejně dostupné sítě apod.) informačních technologií v organizaci? Pomocí jakých prostředků a opatření byste provedl zabezpečení interních informačních systémů v případě, že je chcete chránit před vnějšími i vnitřními hrozbami?

1. Zabezpečení vnějšího perimetru (Ochrana proti Internetu)

Cílem zabezpečení perimetru je vytvořit neprostupnou bariéru mezi nedůvěryhodným internetem a chráněnou firemní sítí.

Next-Generation Firewall (NGFW)

- Hlavní hraniční prvek. Neřídí přístup jen podle IP adres a portů, ale provádí hlubokou inspekci paketů (DPI), identifikuje aplikace a blokuje malware.

DMZ (Demilitarizovaná zóna)

- Izolovaný segment sítě, kde jsou umístěny servery veřejně dostupné z internetu (Web, E-mail, VPN brány).
- Pokud útočník kompromituje server v DMZ, je stále oddělen dalším firewallem od vnitřní sítě (LAN), takže se nedostane k citlivým datům.

VPN a šifrované tunely (IPsec / SSL VPN)

- Zajišťují bezpečný vzdálený přístup pro zaměstnance z domova (Home Office) nebo poboček. Veškerá komunikace zvenčí musí být silně šifrovaná.

WAF (Web Application Firewall)

- Nasazuje se před webové servery v DMZ. Filtruje HTTP komunikaci a detekuje aplikační útoky, které klasický firewall propustí (např. SQL Injection, XSS).

Ochrana proti DDoS

- Využití služeb CDN (např. Cloudflare) nebo scrubbing center poskytovatele internetu k absorpci a odfiltrování obrovského množství falešného provozu předtím, než zahltlí linky organizace.

E-mail Security Gateway

- Antispamový a antiphishingový filtr. Odstraňuje škodlivé přílohy a odkazy dříve, než dorazí do uživatelské schránky.

2. Zabezpečení interních systémů (Před vnějšími i vnitřními hrozbami)

Zabezpečit pouze perimetr už dnes nestačí (např. zaměstnanec si přinese infikovaný flash disk, nebo záměrně škodí jako "insider threat"). Moderním přístupem je **Zero Trust** (nedůvěřuj ničemu, ani interní síti).

Segmentace a Mikrosegmentace sítě (VLAN, interní Firewally)

- Rozdělení ploché interní sítě na menší, logické celky (např. oddělená síť pro účtárnu, pro servery a pro běžné uživatele).
- Komunikace mezi těmito segmenty je filtrována. Zásadně to brání **Laterálnímu pohybu (Lateral Movement)** – útočník, který nakazí PC recepční, se nedostane k databázím.

NAC (Network Access Control / 802.1X)

- Ověřuje každé zařízení před tím, než dostane IP adresu a přístup do vnitřní sítě (LAN i Wi-Fi). Zařízení bez aktuálního antiviru nebo neznámá zařízení jsou přiřazena do izolované sítě pro hosty.

Zabezpečení koncových bodů (EDR/XDR)

- Nasazení pokročilých antivirových řešení na všechny interní stanice a servery. EDR nemonitoruje jen soubory, ale chování procesů a detekuje hrozby přímo na zařízení.

SIEM (Security Information and Event Management)

- Nástroj pro centrální sběr a analýzu logů z vnitřní sítě i perimetru. Odhaluje korelace (např. "Zaměstnanec HR náhle v noci skenuje porty účetního serveru").

IAM (Identity and Access Management) a MFA

- Silné řízení přístupu. Využití Active Directory s uplatněním principu nejnižšího privilegia (PoLP).
- Nasazení vícefázového ověřování (MFA) pro přístup k jakémukoliv internímu informačnímu systému, bez ohledu na to, odkud uživatel přistupuje.

Praktické příklady

1. **DMZ pro veřejný web:** Webový server firmy je v DMZ, protože musí být dostupný z internetu. Databáze s osobními údaji ale zůstává ve vnitřní síti za dalším firewallem, takže kompromitace webu automaticky neznamená přístup k databázi.
2. **Mikrosegmentace uvnitř firmy:** Počítač recepce nemá síťově povolený přístup k účetní databázi. Pokud se nakazí malwarem, interní firewall zabráni laterálnímu pohybu k citlivým serverům.

Zabezpečení webové infrastruktury (Hrozby a prevence)

Představte si, že jste zodpovědný za WEB infrastrukturu organizace. Jaké útoky hrozí, jaké mohou mít důsledky a jak jim zabráníte?

Jako správce webové infrastruktury čelím neustálým pokusům o narušení důvěrnosti, integrity a dostupnosti. Nejčastější hrozby a postupy obrany definuje organizace **OWASP (Open Worldwide Application Security Project)**.

1. Nejvážnější hrozby, důsledky a způsoby prevence

A. SQL Injection (SQLi)

Princip útoků

Útočník vloží do vstupního pole aplikace (formulář, parametr v URL) škodlivý kód SQL. Pokud aplikace tento kód slepě spojí s dotazem, databáze jej vykoná.

Důsledky

Krádež hesel, neautorizované smazání nebo změna databáze, převzetí kontroly nad databázovým serverem.

Prevence

Výhradní použití parametrizovaných dotazů (Prepared Statements)

(nebo ORM frameworků). Data jsou vždy zpracována jako data, nikdy jako spustitelný kód.

- Validace a sanitizace uživatelských vstupů (odstranění nebezpečných znaků).

B. Cross-Site Scripting (XSS)

Princip útoků

Útočník propašuje do stránek (např. do komentáře ve fóru) škodlivý JavaScript. Když si pak stránku otevře jiný uživatel (oběť), kód se v jeho prohlížeči spustí.

Důsledky

Krádež relací (Session Hijacking / krádež cookies), přesměrování oběti na phishing, vykonání akce pod účtem oběti.

Prevence

Escapování / Enkódování

veškerého uživatelského obsahu před jeho vykreslením (např. znak `<` se převede na `<`).

- Nasazení HTTP hlavičky **CSP (Content Security Policy)**, která prohlížeči řekne, odkud přesně smí spouštět skripty.

C. Cross-Site Request Forgery (CSRF)

Princip útoku

Útočník donutí prohlížeč přihlášené oběti vykonat nechtěnou akci (např. posláni peněz, změna e-mailu na profilu) na zranitelném webu bez jejího vědomí.

Důsledky

Neoprávněné změny stavu nebo transakce provedené jménem legitimního uživatele.

Prevence

- Používání **Anti-CSRF tokenů** (náhodný generovaný řetězec, který musí být obsažen v každém POST požadavku a který útočník nemůže znát).
- Nasazení atributu `SameSite` u cookies.

D. Útoky typu odepření služby (DDoS)

Princip útoku

Zahlcení infrastruktury obrovským množstvím požadavků. Buď na síťové vrstvě (zahlcení linky paketovým floodingem), nebo na aplikační vrstvě (miliony HTTP GET požadavků vyčerpávající paměť a CPU serveru).

Důsledky

Úplný výpadek webové aplikace pro legitimní uživatele, finanční ztráty, poškození reputace.

Prevence

- Umístění webu za **Reverzní proxy nebo CDN** (např. Cloudflare, Akamai), která absorbuje zátěž.
- Zavedení **Rate Limitingu** (omezení počtu požadavků z jedné IP adresy za časový úsek). Auto-scaling serverů v cloudu.

E. Man-in-the-Middle (Odposlech komunikace)

Princip útoku

Útočník na stejné síti (např. veřejná Wi-Fi kavárny) odposlouchává nebo mění pakety letící mezi uživatelem a serverem.

Důsledky

Krádež přihlašovacích údajů, bankovních dat nebo podvržení obsahu stránky.

Prevence

- Plošné vynucení **šifrování přes HTTPS (TLS 1.2 nebo 1.3)**.
- Použití hlavičky **HSTS (HTTP Strict Transport Security)**, která prohlížeči zakáže připojovat se nešifrovaně. Zabezpečení certifikátů a privátních klíčů.

2. Další komplexní opatření

Provoz WAF (Web Application Firewall)

Detekuje a blokuje neobvyklé vzory a pokusy o exploitaci přímo v HTTP provozu dříve, než dosáhnou aplikační logiky.

Patch Management

Pravidelné a rychlé aktualizování operačních systémů, webových serverů (Nginx, Apache), databází i použitých frameworků a knihoven.

Bezpečnostní hlavičky a konfigurace

Správné nastavení webového serveru (skrytí verzí SW v hlavičkách `Server` a `X-Powered-By`, zamezení clickjackingu pomocí `X-Frame-Options`).

Řízení oprávnění na serveru

Webový server by měl běžet pod uživatelem s co nejnižším možným oprávněním (určitě ne jako root/administrator) a neměl by mít přístup k celému filesystému.

Praktické příklady

- 1. SQL Injection v přihlášení:** Pokud aplikace skládá dotaz jako text `SELECT * FROM users WHERE name = ' + vstup + '`, útočník může vložit škodlivý SQL fragment. Prepared statements tomu zabrání, protože vstup se bere jako data, ne jako část SQL příkazu.
- 2. XSS v komentářích:** Útočník vloží do komentáře JavaScript, který kradе session cookie. Aplikace se brání escapováním výstupu, nastavením `HttpOnly` u cookies a CSP hlavičkou, která omezuje spouštění skriptů.